# Anveshana Club

## Specification Audit, Conflict Resolution & Implementation Corrections

*"Explore · Build · Bring Ideas to Life."*

**Institution:** Mohan Babu University, Tirupati

**Document Type:** Master Audit & AI Agent Supplementary Specification

## 1. Document Inventory and Audit Scope

**Documents Reviewed:** 1. Product Requirements Document (PRD) 2. Technical Requirements Document (TRD) 3. Application Flow & Navigation 4. UI/UX Design Brief 5. Backend Schema & API Specification 6. Implementation Plan & Roadmap 7. Native Event Registration & Form Builder Change Request (CR)

**Audit Scope:** This document resolves contradictions between the baseline specifications (Docs 1-6) and the Native Registration Change Request (Doc 7). It establishes canonical implementation decisions, architectural corrections, security baseline standards, and strict operational instructions for the AI coding agent. It does not replace the original documents but serves as the absolute authority wherever conflicts exist.

---

## 2. Conflict Register

| ID | Conflict Area | Conflicting Statements | Risk | Recommended Resolution | Human Approval? |
|---|---|---|---|---|---|
| **CONF-01** | Event Registration | PRD/TRD mentions Google Forms/ WhatsApp redirection; CR requires Native website registration. | Critical | **CR supersedes.** Complete removal of external form links. Event registration must occur natively via the dynamic form builder. | Safe to Apply |

| ID | Conflict Area | Conflicting Statements | Risk | Recommended Resolution | Human Approval? |
|---|---|---|---|---|---|
| **CONF-02** | Event Routing | Flow Doc states `/events/:id`; PRD implies SEO-friendly URLs (`/events/:slug`). | Medium | Public routes use `/events/:slug` for SEO. Admin backend CRUD uses `_id`. | Safe to Apply |
| **CONF-03** | Feedback/ Contact | PRD lists separate pages; App Flow combines them. | Low | Unify under `/contact`. Submit feedback via `/api/v1/feedback`. | Safe to Apply |
| **CONF-04** | Event Status | PRD implies simple Open/ Closed; CR requires Upcoming, Open, Closed, Full, Draft, Archived. | High | Standardize `Event` document status to `Draft`, `Published`, `Archived`. Registration state (Upcoming, Open, Closed, Full) is **computed dynamically** based on dates and capacity. | Safe to Apply |
| **CONF-05** | Team Categories | Schema Doc lists an Enum; PRD implies an editable collection. | Medium | Use a strict Enum in Mongoose for MVP (`Team Leads`, `Infrastructure`, `Design`, etc.) to prevent UI breakage. | Safe to Apply |
| **CONF-06** | Data Deletion | Schema implies hard deletes; CR requires historical preservation. | High | Implement **Soft Deletion** (`isDeleted: true`) on Events, Forms, and Registrations. No hard deletes. | Safe to Apply |
| **CONF-07** | Galleries | PRD implies galleries per event; Schema ambiguous on nesting vs references. | Medium | Use embedded `gallery` array (Cloudinary URLs) within the Event document for MVP simplicity. | Safe to Apply |
| **CONF-08** | API Conventions | Inconsistent prefixing across TRD and Schema doc. | High | Strict enforcement of `/api/v1/` prefix and JSend response format (`{ status, data, message }`). | Safe to Apply |

| ID | Conflict Area | Conflicting Statements | Risk | Recommended Resolution | Human Approval? |
|---|---|---|---|---|---|
| **CONF-09** | Deduplication | CR requires deduplication; baseline missing unique constraints. | High | Deduplicate via unique compound index on `{ eventId, studentId }` (if studentId is required for the event). | Safe to Apply |
| **CONF-10** | Roadmap Scope | Original plan lacks Form Builder; CR tasks appended arbitrarily. | High | Integrated roadmap defined in Section 7 of this document. | Safe to Apply |

---

## 3. Canonical Implementation Decisions

1. **Routing:**
2. Public: `/`, `/events`, `/events/:slug`, `/team`, `/contact`.
3. Admin: `/admin/login`, `/admin/dashboard`, `/admin/events`, `/admin/events/:id/participants`.
4. **API Design:** All endpoints prefixed with `/api/v1/`. Standardized envelope: `{ status: "success"|"error", data: any, message: string }`.
5. **Event Lifecycle:**
6. Database Status: `Draft`, `Published`, `Archived`.
7. Registration State (Computed): `Upcoming` (now < openDate), `Open` (openDate <= now <= closeDate AND registrations < capacity), `Full` (registrations >= capacity), `Closed` (now > closeDate).
8. **Timezones:** All dates stored in UTC. Converted to IST (Asia/Calcutta) on the frontend.
9. **Authentication:** Single Admin role. JWT stored in `HttpOnly`, `Secure`, `SameSite=Strict` cookies. No public student accounts.
10. **Data Retention:** Soft deletion pattern. Modifying event forms must not orphan or alter historical registration data.

---

## 4. Native Registration Architecture Corrections

**Dynamic Form Builder & Schema Versioning:** - **Form Schemas:** Stored as an array of schema objects within the Event document. - **Versioning:** When an admin publishes a form

change, a new schema object is pushed to the array with a unique `versionId` and `createdAt` timestamp. - **Stable IDs:** Fields utilize auto-generated stable UUIDs (e.g., `field_uuid_123`). Renaming a label updates the current schema version but retains the UUID. - **Data Binding:** Every `Registration` document must store the `formVersionId` it was submitted against.

**Concurrency & Participant Limits (Atomic Operations):** - Do not assume multi-document MongoDB transactions are configured. - To prevent overselling, use atomic updates: `Event.findOneAndUpdate({ _id: eventId, registeredCount: { $lt: capacityLimit } }, { $inc: { registeredCount: 1 } })`. - If this returns null, the event is full, and the registration must be rejected *before* saving the participant record.

**Validation & Submission:** - The backend must fetch the **active** published form schema for the event and strictly validate incoming payload keys against valid field UUIDs and data types. - Reject unknown fields, altered dropdown choices, or bypassed required fields. - Generate a unique 8-character alphanumeric `referenceNumber` upon successful database persistence.

**Participant Management & Export:** - **Table View:** Fetch registrations by `eventId`. Flatten the JSON answers based on the active form schema for display. - **CSV Injection Prevention:** When generating CSV/XLSX, prepend a single quote (`'`) to any submitted value starting with `=`, `+`, `-`, or `@` to prevent spreadsheet formula injection.

---

## 5. Security and Privacy Corrections

1. **Authentication & Passwords:** Admin accounts provisioned directly via DB script (no public registration). Passwords hashed with `bcrypt` (12 salt rounds).
2. **Session Management:** Provide an `/api/v1/auth/logout` endpoint that clears the HttpOnly cookie.
3. **Rate Limiting:** Apply `express-rate-limit` to `/api/v1/auth/login` (max 5 requests/15 mins) and `/api/v1/registrations` (max 3 requests/minute per IP).
4. **CORS:** Restrict CORS origins strictly to the frontend application URL in production.
5. **File Uploads:** Cloudinary uploads restricted to `image/jpeg`, `image/png`, `image/webp`. Max size 5MB. Implement an orphaned asset cleanup script or overwrite strategy.
6. **Privacy:** Public event endpoints MUST NOT expose participant arrays, emails, or personal data. Participant data is exclusively available on authenticated `/admin/*` routes.
7. **Secrets:** All URI strings, JWT secrets, and Cloudinary keys remain in `.env` and are strictly excluded from source control.

---

## 6. Admin Content Management Boundaries

**What is Editable via Admin Dashboard:** - Event details, publication status, dates, banners, and embedded galleries. - Event-specific registration form configurations (Add/Edit/Remove/Reorder fields). - Participant viewing, filtering, and exporting. - Reviewing and soft-deleting feedback submissions. - Managing Core Team member profiles.

**What Requires Code/DB Intervention (Not in Admin UI MVP):** - Global website branding (colors, typography, tagline). - Changing the predefined Team Categories list. - Creating new Administrator accounts. - Homepage static copy.

---

## 7. Corrected Implementation Roadmap

**Phase 1: Baseline Setup & Security** - Inspect repo, setup Node/Express + TS, React + Vite. Configure MongoDB Mongoose schemas with soft deletes. Implement JWT HttpOnly Auth. **Phase 2: Public UI & Core CRUD** - Build public event listings, team pages, and contact form. Implement Admin CRUD for Events, Teams, and Feedback. **Phase 3: Dynamic Form Builder (Backend & Admin)** - Implement schema versioning in Event models. Build React drag-and-drop or standard UI for form builder. **Phase 4: Native Registration & Concurrency** - Public dynamic form renderer. Backend validation, atomic capacity checks, and reference number generation. **Phase 5: Participant Management** - Admin registration tables, detail views, pagination, and secure CSV export (with formula injection mitigation). **Phase 6: Testing, Polish & Handover** - Execute acceptance matrix, mobile responsiveness checks, and Cloudinary error handling.

---

## 8. Test and Acceptance Matrix

| Test ID | Scenario | Expected Result | Severity |
| --- | --- | --- | --- |
| TEST-01 | Unauthenticated access to `/api/v1/admin/*` | 401 Unauthorized, no data leaked. | Critical |
| TEST-02 | Submit registration for `Draft` event | 400 Bad Request, rejected by backend. | High |
| TEST-03 | Alter dropdown choice in browser inspector | Backend rejects payload due to invalid choice. | High |
| TEST-04 | Concurrent submissions at capacity limit | Exactly capacity limit reached; overflow requests receive 'Event Full' error. | Critical |

| Test ID | Scenario | Expected Result | Severity |
| --- | --- | --- | --- |
| TEST-05 | Duplicate registration (same event, same student ID) | Rejected by backend unique index/validation rule. | High |
| TEST-06 | Registration post-deadline | 400 Bad Request, 'Registration Closed'. | High |
| TEST-07 | CSV Export with payload `=cmd\|' /C calc'!A0` | Output sanitized as `'=cmd\|' /C calc'!A0`, executes as raw text in Excel. | Critical |
| TEST-08 | Public API fetches Event details | Does not return `registrations` or `formSchemas` history. | High |
| TEST-09 | Edit active form with existing responses | New form version created; old registrations remain intact and readable. | High |

---

## 9. AI Coding-Agent Operating Instructions

**Strict Directives for Antigravity (AI Agent):** 1. **Source of Truth:** Read all 7 original documents AND this audit document before executing any code changes. This document takes ultimate precedence. 2. **Do Not Overwrite Unrelated Files:** Inspect the existing repository state using terminal tools before creating or modifying files. 3. **No Mock Data for Production:** Implement real MongoDB Mongoose queries and Express routes. Do not use local JSON arrays to simulate database persistence. 4. **Iterative Execution:** Work ONE phase at a time. Run `npm run build` or relevant type-checks after each phase. 5. **Reporting:** After each step, report files changed, commands executed, actual results, unresolved issues, and next steps. 6. **Pause for Approval:** Stop and explicitly request human approval before executing destructive migrations or if a product decision is undocumented. 7. **No Credential Hardcoding:** Always use `process.env` for DB URIs, Secrets, and API Keys. 8. **Document Preservation:** NEVER modify, delete, or rewrite the original 7 specification documents.

---

## 10. Final Decision Summary

- **Critical Blockers Resolved:** Native registration mandated. Atomic capacity updates specified. CSV injection prevented.
- **High-Priority Corrections:** Implementation of Schema Versioning for forms to protect historical data. Separation of DB Status (`Published`) from computed Registration State (`Open/Closed`).
- **Owner Decisions Required:** None immediately blocking MVP. Future requirement needed for Team Categories (Enum vs Editable Collection).

- **First Implementation Task:** Initialize repository, connect MongoDB Atlas, create standard Mongoose schemas (Event, Team, Feedback, Admin) incorporating `isDeleted` flags, and configure JWT authentication scaffolding.